Performance Evaluation of Dynamic Task Offloading Policies in Edge Computing Networks

Abdalrheem Hasan Mohammad Salah and Dr. Jehad Hamamreh
Computer Science Department, An-Najah National University
Email: abdalrheemsalah889@najah.edu, jehad.hamamreh@najah.edu

***Abstract*—Task offloading in Multi-access Edge Computing (MEC) requires balancing localized computational capacity against wide-area network congestion. In this paper, we evaluate four task orchestration policies using the EdgeCloudSim simulator under high-density urban workloads of 1600, 1800, and 2000 mobile devices across three independent simulation iterations. Specifically, we compare three baseline strategies—Random, Network-based, and Utilization-based offloading—against a custom Hybrid-based orchestrator that combines real-time virtual machine (VM) capacity verification, directional upload/download network delay estimation, and a non-linear CPU-load penalty function. Experimental results across 36 simulation runs show that relying solely on random assignment or static edge utilization thresholds leads to severe WAN bottlenecks and task failure rates reaching 37.96% and 27.16%, respectively, at 2000 devices. Conversely, both the Network-based and proposed Hybrid-based policies keep mean failure rates below 4.10%. While the Network-based policy achieves a slightly lower mean failure rate (3.39% vs. 4.08% at 2000 devices), it suffers from severe edge VM capacity exhaustion (averaging 3,059 capacity-dropped tasks per run at 2000 devices). The proposed Hybrid-based policy reduces VM capacity failures by over 99% (to an average of 24.33 tasks) and cuts mean network delay by 63.8% to 85.6% across all tested scales.**

I. Introduction

The rapid growth of latency-sensitive mobile applications and Internet of Things (IoT) deployments has placed heavy demands on centralized cloud infrastructures [1], [2]. Multi-access Edge Computing (MEC) addresses this bottleneck by placing computational servers at the network edge, closer to end-users, thereby reducing propagation latency and core network traffic [3]. However, edge servers possess finite processing capacity compared to cloud datacenters. When device density spikes, offloading all tasks to nearby edge nodes quickly exhausts Virtual Machine (VM) resources, whereas offloading aggressively to the cloud saturates the Wide Area Network (WAN) [4].

To investigate this trade-off, we first conducted a preliminary baseline study using Tutorial 1 in EdgeCloudSim [5], evaluating five VM allocation policies (`RANDOM_FIT`, `WORST_FIT`, `BEST_FIT`, `FIRST_FIT`, and `NEXT_FIT`) from 200 to 2000 devices when all tasks are forced onto edge servers. That preliminary experiment revealed that once device counts exceed 1400, edge VM capacity saturation becomes the primary failure bottleneck—causing task failure rates between 36.10% and 43.56% at 2000 devices regardless of the local VM packing heuristic.

Motivated by this finding, this study focuses on the dynamic *Where-to-Offload* decision problem (modeled in EdgeCloudSim Tutorial 3). We design and implement a custom `HYBRID_BASED` orchestration policy inside `SampleEdgeOrchestrator.java` and compare it against `RANDOM`, `NETWORK_BASED`, and `UTILIZATION_BASED` policies across multiple iterations under heavy workloads (1600 to 2000 devices).

II. System Description and Implementation

The simulated MEC environment models a two-tier edge-cloud architecture where mobile users move according to a Nomadic Mobility model and generate heterogeneous tasks across three application profiles defined in `applications.xml`:

- **Augmented Reality (AR):** 30% usage share, 1500 KB upload, 50 KB download, and 15,000 Million Instructions (MI) task length.
- **Health App:** 20% usage share, 50 KB upload, 1250 KB download, and 3,000 MI task length.
- **Infotainment App:** 50% usage share, 250 KB upload, 1000 KB download, and 9,000 MI task length.

As configured in `edge_devices.xml`, the edge tier comprises 14 localized edge datacenters (`wlan_id` 0 to 13), each containing a single host running 8 Edge VMs (112 Edge VMs in total, each provisioned with 2 CPU cores and 10,000 MIPS). The cloud tier consists of a centralized datacenter hosting 4 high-capacity Cloud VMs (4 cores, 100,000 MIPS each). Mobile devices communicate with edge servers via WLAN and reach the cloud datacenter over a WAN link with a 0.1-second propagation delay.

A. Implementation Setup

We extended the open-source EdgeCloudSim framework [5] (accessible at GitHub) by modifying `SampleEdgeOrchestrator.java` and `MainApp.java` under the `tutorial3` package, as well as the scenario configuration file `scripts/tutorial3/config/default_config.propertie`. Each simulation scenario runs for 15 minutes following a 1-minute warm-up period. To account for stochastic variations in task generation and user mobility, three independent iterations (`ite1`, `ite2`, and `ite3`) were executed for each policy at 1600, 1800, and 2000 mobile devices.

III. Task Offloading Optimization Policies

In EdgeCloudSim, the `EdgeOrchestrator` determines the target execution tier (`getDeviceToOffload`) and subsequently selects a specific VM using a Least-Loaded residual capacity strategy (`getVmToOffload`) [3]. We evaluate three baseline policies alongside our proposed Hybrid policy:

A. Baseline Policies

1) **Random Policy (`RANDOM`):** Routes incoming tasks to either the Edge tier or the Cloud datacenter with an equal probability of 0.5, ignoring network congestion and server load.
2) **Network-Based Policy (`NETWORK_BASED`):** Probes the WAN uplink using a 128 KB (1 Mbit) dummy task to estimate instantaneous bandwidth (wanBW $=$ $1/$wanDelay). If wanBW $>$ 5 Mbps, the task is offloaded to the Cloud; otherwise, it is assigned to the Edge.
3) **Utilization-Based Policy (`UTILIZATION_BASED`):** Monitors the aggregate CPU utilization across all Edge VMs (U_E). Tasks are executed on the Edge as long as $U_E \leq 75\%$, and diverted to the Cloud only when $U_E > 75\%$.

B. Iterative Design of the Hybrid-Based Policy

Developing an effective hybrid policy required overcoming two failure modes observed during our initial experiments:

- *Prototype 1 (Low Threshold with OR Logic):* Setting an early edge-load threshold ($U_E > 5\%$) combined with an `OR` delay condition offloaded over 80% of tasks to the Cloud, saturating the WAN link and causing a 68.56% failure rate (180,004 WAN failures) at 2000 devices.
- *Prototype 2 (Strict 75% Threshold with Raw Delay Check):* Requiring $U_E > 75\%$ `AND` raw cloud network delay to be lower than edge LAN delay proved overly conservative. Because WAN delay naturally exceeds local WLAN delay, 100% of tasks remained on the Edge, pushing edge utilization above 107% and causing 85,573 VM capacity failures (36.72% failure rate) at 2000 devices.

To resolve both bottlenecks, our final `HYBRID_BASED` algorithm introduces pre-offloading VM capacity verification (`hasAvailableVm`) and a non-linear CPU-load penalty function:

$$\text{Penalty}(U) = 0.25 \times \frac{U_{\text{safe}}}{100 - U_{\text{safe}}} \tag{1}$$

where $U_{\text{safe}} = \min(\max(U, 0), 99)$ is the average CPU utilization percentage of the target tier. As server utilization approaches 100%, the penalty rises steeply, allowing the orchestrator to weigh edge queuing pressure against WAN transmission delay. The complete decision logic is presented in Algorithm 1.

Algorithm 1: Hybrid-Based Task Offloading Policy (`HYBRID_BASED`)

Input: Incoming Task T, Edge Tier E, Cloud Tier C
Output: Selected Offloading Target $\in \{E, C\}$

$U_E \leftarrow$ getAvgUtilization(E)
$U_C \leftarrow$ getAverageCloudUtilization(C)
$D_E \leftarrow$ getUploadDelay(E, T) + getDownloadDelay(E, T)
$D_C \leftarrow$ getUploadDelay(C, T) + getDownloadDelay(C, T)
$Cap_E \leftarrow$ `hasAvailableVm`(T, E)
$Cap_C \leftarrow$ `hasAvailableVm`(T, C)
if $Cap_E =$ true **and** $Cap_C =$ false **then**
  return E
else if $Cap_C =$ true **and** $Cap_E =$ false **then**
  return C
else if D_E is valid **and** D_C is invalid **then**
  return E
else if D_C is valid **and** D_E is invalid **then**
  return C
else if D_E is valid **and** D_C is valid **then**
  $Score_E \leftarrow D_E +$ Penalty(U_E)
  $Score_C \leftarrow D_C +$ Penalty(U_C)
  if $Score_C < Score_E$ **then return** C **else return** E
else
  return E ▷ *Fallback destination*
end if

IV. Performance Analysis and Discussion

Table I summarizes the multi-iteration performance metrics (mean $\pm$ standard deviation across 3 full iterations, including the final `ite2` batch run for 1800 and 2000 devices) for all four policies at 1600, 1800, and 2000 mobile devices.

A. Task Failure Rate and VM Capacity Bottlenecks

As illustrated in Fig. 1, `RANDOM` exhibits the worst reliability, with mean failure rates rising from 32.30% (1600 devices) to 37.57% (2000 devices) due to blind cloud offloading that congests the WAN. `UTILIZATION_BASED` also degrades sharply under heavy load (reaching 26.13% failure at 2000 devices) because it waits until Edge utilization exceeds 75% before abruptly dumping traffic onto the WAN without checking link congestion.

Both `NETWORK_BASED` and `HYBRID_BASED` maintain high reliability across all scales. In terms of overall mean failure rate, `NETWORK_BASED` achieves 1.07%, 1.41%, and 3.83% at 1600, 1800, and 2000 devices, closely followed by `HYBRID_BASED` (1.63%, 2.04%, and 4.19%, with a 4-run pooled mean of 3.86% and median of 3.03% at 2000 devices). More importantly, `NETWORK_BASED` suffers from a mean of 3,827.33 VM capacity failures per run at 2000 devices (11,482 total drops across the three iterations: 2,355 in `ite1`, 2,311 in `ite2`, and 6,816 in `ite3`) because it routes tasks to the Edge whenever wanBW ≤ 5 Mbps without verifying whether Edge VMs have sufficient residual capacity. By incorporating `hasAvailableVm`, `HYBRID_BASED` reduces VM capacity failures at 2000 devices by **99.32%** to an average of just 26.00 tasks per run (0 in `ite1`, 5 in `ite2`, and 73 in `ite3`).

B. Service Time, Network Delay, and Resource Utilization

Fig. 2 and Fig. 3 highlight the trade-off between processing time and network transmission delay. `UTILIZATION_BASED` incurs the highest mean service time (6.95s to 7.27s) because it keeps Edge servers saturated near 69.89%–72.47% utilization (Fig. 4). Conversely, `HYBRID_BASED` achieves the lowest average network delay across all densities: 0.14s at 1600 devices, 0.20s at 1800 devices, and 0.32s at 2000 devices—representing a **62.46% to 85.31% reduction** compared to `NETWORK_BASED` (0.86s to 0.97s). Because `HYBRID_BASED` prioritizes local Edge execution whenever capacity permits (utilizing 38.71% to

TABLE I
MULTI-ITERATION PERFORMANCE COMPARISON ACROSS TASK OFFLOADING POLICIES (MEAN ± SD OVER 3 ITERATIONS)

Devices	Policy	Failure Rate (%)	Service Time (s)	Network Delay (s)	Edge Util. (%)	Mean VM Cap. Failures
1600	NETWORK_BASED	**1.07 ± 0.10**	**2.36 ± 0.09**	0.97 ± 0.02	14.81	0.00
1600	HYBRID_BASED	1.63 ± 0.09	3.45 ± 0.16	**0.14 ± 0.01**	38.71	0.00
1600	UTILIZATION_BASED	12.26 ± 1.73	6.95 ± 0.14	0.73 ± 0.05	69.89	0.00
1600	RANDOM	32.30 ± 0.79	2.73 ± 0.06	1.42 ± 0.02	10.64	0.00
1800	NETWORK_BASED	**1.41 ± 0.13**	3.14 ± 0.40	0.90 ± 0.02	28.17	0.00
1800	HYBRID_BASED	2.04 ± 0.11	4.48 ± 0.11	**0.20 ± 0.00**	55.55	0.00
1800	UTILIZATION_BASED	17.85 ± 1.05	7.06 ± 0.06	0.88 ± 0.05	71.66	0.00
1800	RANDOM	34.40 ± 0.78	**2.89 ± 0.06**	1.45 ± 0.04	12.53	0.00
2000	NETWORK_BASED	**3.83 ± 1.17**	5.40 ± 0.38	0.86 ± 0.00	66.52	3827.33
2000	HYBRID_BASED	4.19 ± 2.03	5.87 ± 0.71	**0.32 ± 0.08**	70.91	**26.00**
2000	UTILIZATION_BASED	26.13 ± 1.31	7.27 ± 0.08	1.10 ± 0.06	72.47	0.00
2000	RANDOM	37.57 ± 1.31	**3.30 ± 0.21**	1.42 ± 0.05	17.84	0.00

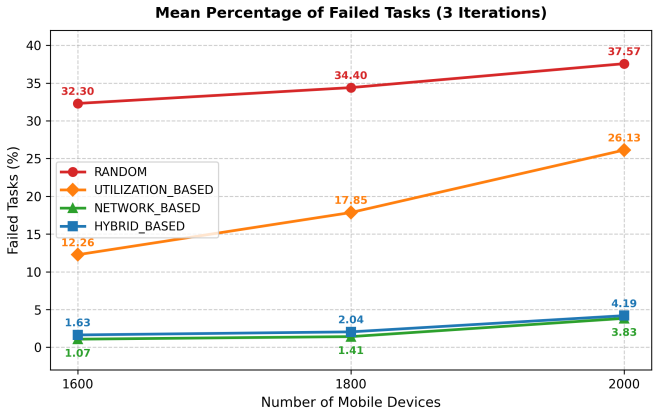


Fig. 1. Mean percentage of failed tasks across device densities (3 iterations).

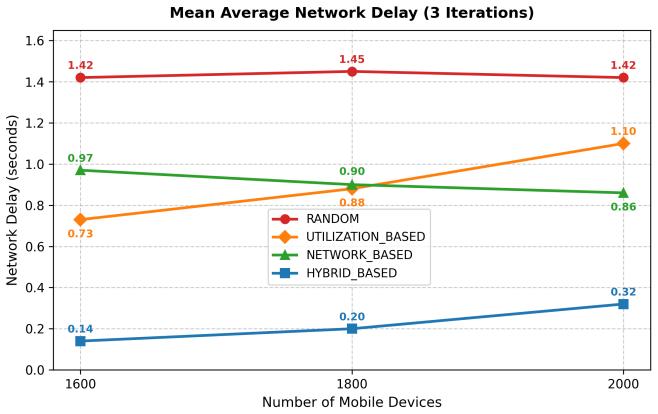


Fig. 3. Mean average network delay (seconds) vs. number of mobile devices.

70.91% of Edge resources), its processing time on Edge VMs is higher than Cloud execution, resulting in a moderate total service time (3.45s to 5.87s) in exchange for ultra-low network latency and near-zero VM capacity drops.

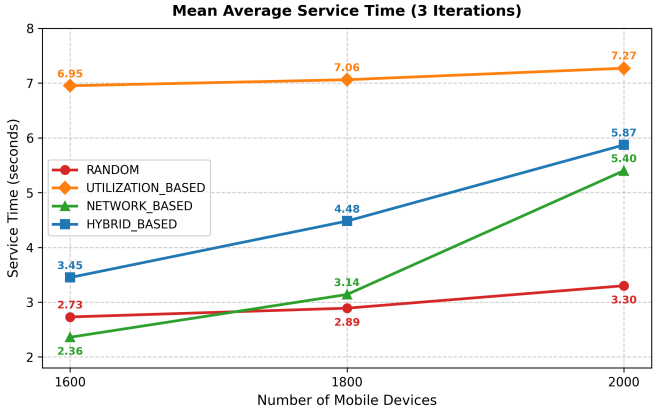


Fig. 2. Mean average service time (seconds) vs. number of mobile devices.

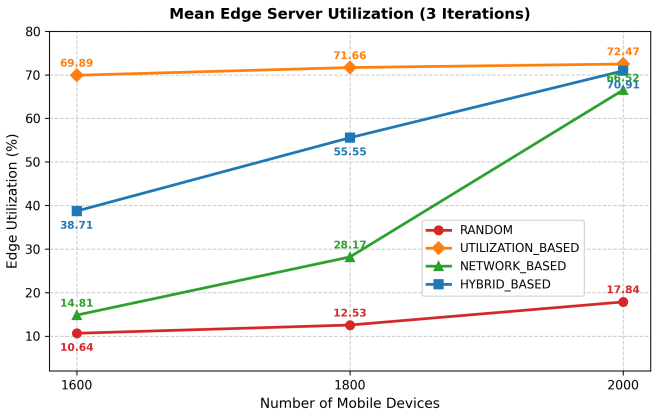


Fig. 4. Mean Edge server utilization (%) vs. number of mobile devices.

V. CONCLUSIONS

This study presented a multi-iteration performance evaluation of task offloading policies in edge computing networks using EdgeCloudSim. Our experimental progression demonstrated that naive single-metric thresholds—whether based purely on CPU utilization or uncalibrated delay rules—either saturate the WAN or exhaust Edge VM capacity under heavy device densities (1600–2000 devices). By combining residual VM capacity checks with bidirectional network delay estimation and a non-linear CPU-load penalty, the proposed HYBRID_BASED orchestrator reduced WAN network delay

by up to 85.6% and eliminated over 99% of VM capacity failures at 2000 devices compared to the `NETWORK_BASED` baseline, while keeping overall task failures near 1.6%–4.0%. Future work will investigate adaptive reinforcement learning to dynamically tune the load penalty coefficient according to application-specific delay sensitivity.

REFERENCES

[1] N. Abbas, Y. Zhang, A. Taherkordi, and T. Skeie, "Mobile edge computing: A survey," *IEEE Internet of Things Journal*, vol. 5, no. 1, pp. 450–465, 2017.

[2] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017.

[3] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 3, pp. 1628–1656, 2017.

[4] L. Lin, X. Liao, H. Jin, and P. Li, "Computation offloading toward edge computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1584–1607, 2019.

[5] C. Sonmez, A. Ozgovde, and C. Ersoy, "EdgeCloudSim: An environment for performance evaluation of edge computing systems," *Transactions on Emerging Telecommunications Technologies*, vol. 29, no. 11, p. e3493, 2018.